# Phase 9 Completion Report: OpenMetadata Integration

**1. Phase Overview**

Phase 9 of the Banking Data Governance & Data Quality Pipeline focused on introducing OpenMetadata concepts as an open-source governance and data catalog platform.

The purpose of this phase was not to replace the governance artifacts created in Phase 8, but to demonstrate how those governance artifacts could be represented, cataloged, and managed within a governance platform.

Phase 9 used OpenMetadata as an open-source counterpart for practicing governance concepts commonly associated with enterprise platforms such as Collibra.

This phase was intentionally designed as a metadata integration and governance-platform planning phase. No pipeline execution logic, ingestion logic, data modeling logic, or data quality logic was modified.

The completed pipeline before Phase 9 already included:

PostgreSQL Banking Source System

→ Python Ingestion Layer

→ Raw Parquet Zone

→ PySpark Standardized Layer

→ Curated Warehouse Layer

→ SQL Data Quality Validation Layer

→ PySpark Data Quality Validation Layer

→ Governance Artifacts

Phase 9 added a governance-platform integration layer on top of those completed components.

---

**2. Phase 9 Objective**

The objective of Phase 9 was:

To demonstrate how governance assets, glossary terms, ownership information, lineage, classifications, and metadata from the banking pipeline could be represented and managed within OpenMetadata.

The phase focused on:

- OpenMetadata integration planning
- Metadata ingestion design
- Governance asset mapping

- Catalog representation
- Ownership and stewardship concepts
- Classification and tagging concepts
- Lineage representation
- Governance platform operating concepts

Phase 9 intentionally avoided changing the technical behavior of the data pipeline.

---

**3. Phase 9 Scope**

Phase 9 was divided into three sub-phases:

- Phase 9A – OpenMetadata Integration Design
- Phase 9B – Metadata Ingestion Configuration Templates
- Phase 9C – Local Deployment and Execution Runbook

This phased approach kept the project stable and prevented governance tooling from affecting the existing pipeline.

**In Scope**

Phase 9 included:

- OpenMetadata documentation
- OpenMetadata governance mapping
- Metadata ingestion templates
- Ownership and stewardship examples
- Classification and tagging concepts
- Lineage integration concepts
- Local deployment planning
- Governance platform documentation

**Out of Scope**

The following items were intentionally excluded:

- Changes to PostgreSQL schema
- Changes to Faker data generation
- Changes to Python ingestion

- Changes to Spark processing
- Changes to curated models
- Changes to SQL DQ validation
- Changes to PySpark DQ validation
- Production OpenMetadata deployment
- OpenMetadata installation execution
- Runtime ingestion execution
- Agile/JIRA simulation
- Phase 10 governance operations work

---

**4. Phase 9A – OpenMetadata Integration Design**

**Goal**

The goal of Phase 9A was to establish the architectural approach for OpenMetadata integration.

The work focused on documenting how OpenMetadata concepts align with the governance artifacts created in Phase 8.

**Files Created**

- openmetadata/README.md

**Work Completed**

The OpenMetadata README documented:

- Purpose of OpenMetadata in the project
- Relationship between OpenMetadata and governance artifacts
- Governance-platform integration strategy
- Deployment approach options
- Metadata ingestion approach
- Future lineage integration approach
- Ownership and stewardship concepts

**Governance Mapping**

The following mappings were documented:

**Business Glossary**

OpenMetadata Glossary

↔ governance/glossary/business_terms.yml

**Data Catalog**

OpenMetadata Assets and Tables

↔ governance/catalog/data_assets.yml

**Critical Data Elements**

OpenMetadata Tags and Classifications

↔ governance/metadata/critical_data_elements.yml

**Ownership and Stewardship**

OpenMetadata Owners

↔ Governance Owner and Steward Definitions

**Lineage**

OpenMetadata Lineage Graph

↔ governance/lineage/lineage_mapping.yml

**Data Quality**

OpenMetadata Data Quality Concepts

↔ Phase 6 SQL Validation

↔ Phase 7 PySpark Validation

**Phase 9A Commit**

f790a74

Phase 9A - Document OpenMetadata integration approach

---

**5. Phase 9B – Metadata Ingestion Configuration Templates**

**Goal**

The goal of Phase 9B was to create safe metadata-ingestion configuration templates without requiring a running OpenMetadata environment.

The objective was to practice governance configuration design while avoiding infrastructure dependencies.

**Files Created**

- openmetadata/ingestion_configs/postgres_metadata_ingestion.yml
- openmetadata/ingestion_configs/curated_layer_metadata_ingestion.yml
- openmetadata/ingestion_configs/lineage_ingestion.yml

**Work Completed**

**PostgreSQL Metadata Ingestion Template**

A template was created to represent metadata ingestion for the banking PostgreSQL source system.

The template documented:

- Source connection structure
- Database metadata ingestion concepts
- Environment variable usage
- Secure credential placeholder usage

**Curated Layer Metadata Template**

A template was created to represent future catalog registration for curated warehouse assets.

Examples included:

- Dimension assets
- Fact assets
- Curated warehouse metadata representation

The template was intentionally documentation-oriented and did not require cloud storage migration.

**Lineage Template**

A lineage ingestion template was created to represent lineage registration concepts.

The template aligned with:

- Source-to-target mappings
- Lineage mappings
- Governance documentation from Phase 8

**Security Controls**

Phase 9B enforced the following:

- No passwords committed
- No tokens committed
- No secrets committed

- Environment-variable placeholders only

**Phase 9B Commit**

bb8fbae

Phase 9B - Add OpenMetadata ingestion config templates

---

**6. Phase 9C – Local OpenMetadata Runbook**

**Goal**

The goal of Phase 9C was to document how OpenMetadata could be deployed and used locally in the future.

This phase was documentation-only.

No installation was performed.

No ingestion jobs were executed.

**Files Created**

- openmetadata/local_runbook.md

**Files Updated**

- openmetadata/README.md

**Work Completed**

The runbook documented:

**Prerequisites**

- Docker requirements
- OpenMetadata requirements
- PostgreSQL connectivity requirements
- Environment variable requirements

**Deployment Sequence**

A recommended execution flow was documented:

1. Start existing PostgreSQL services
2. Deploy OpenMetadata separately
3. Configure metadata connections
4. Execute metadata ingestion

5. Validate catalog registration
6. Validate lineage visibility
7. Validate glossary and classifications

**Validation Checklist**

The runbook included validation guidance for:

- Catalog assets
- Business glossary terms
- Ownership assignments
- Stewardship assignments
- Tags and classifications
- Lineage visualization
- Metadata completeness

**Troubleshooting Guidance**

The runbook included guidance for:

- Connection failures
- Missing assets
- Metadata ingestion issues
- Lineage visibility issues

**Phase 9C Commit**

7283746

Phase 9C - Add OpenMetadata local runbook

---

**7. Phase 9 Validation**

Phase 9 validation confirmed the following:

- OpenMetadata documentation created successfully
- Metadata-ingestion templates created successfully
- Local deployment runbook created successfully
- Governance mappings documented successfully
- Environment variable placeholders used

- No secrets committed
- No runtime files generated
- No Docker changes made
- No PostgreSQL changes made
- No pipeline logic modified
- No SQL validation logic modified
- No PySpark validation logic modified
- Git working tree remained clean after completion

---

**8. Final Git History**

Phase 9 commits:

f790a74 Phase 9A - Document OpenMetadata integration approach

bb8fbae Phase 9B - Add OpenMetadata ingestion config templates

7283746 Phase 9C - Add OpenMetadata local runbook

Tag created:

phase-9-complete

---

**9. Skills Practiced in Phase 9**

Phase 9 helped practice:

- Data catalog concepts
- Metadata management
- Governance platform integration
- OpenMetadata architecture
- Collibra-style governance concepts
- Metadata ingestion configuration
- Business glossary integration
- Ownership and stewardship management
- Data classification and tagging
- Data lineage representation

- Governance platform documentation
- Secure configuration management
- Governance operating concepts
- SDLC-based phased delivery
- Git commit and tagging discipline

---

**10. Governance Relevance**

Phase 9 is important because governance programs require more than technical pipelines.

Organizations need governance platforms that provide:

- Business glossaries
- Data catalogs
- Data ownership visibility
- Stewardship accountability
- Data classifications
- Lineage visibility
- Metadata discovery
- Governance workflows

Phase 9 demonstrated how the governance artifacts created in Phase 8 could be represented inside a governance platform.

This phase connected:

- Business terms
- Critical Data Elements
- Data assets
- Ownership assignments
- Stewardship responsibilities
- Lineage mappings
- Data quality concepts

to a practical governance platform architecture.

---

**11. Interview Explanation**

A concise explanation of Phase 9 would be:

“In Phase 9, I integrated OpenMetadata concepts into my banking data governance pipeline. The earlier phases had already established source ingestion, PySpark transformations, curated warehouse models, SQL data quality validation, PySpark data quality validation, and governance documentation. In Phase 9, I documented how those governance assets would be represented within OpenMetadata.

I created OpenMetadata integration documentation, metadata-ingestion configuration templates, and a local deployment runbook. I mapped business glossary terms, catalog assets, Critical Data Elements, ownership assignments, classifications, lineage mappings, and data quality concepts from the governance artifacts into OpenMetadata concepts.

I intentionally kept the phase documentation-driven and configuration-driven. I did not modify pipeline logic, data models, validation logic, or source systems. The goal was to practice governance platform integration concepts while maintaining a stable data engineering pipeline.”

---

**12. Phase 9 Conclusion**

Phase 9 successfully introduced governance-platform integration concepts into the Banking Data Governance & Data Quality Pipeline.

The project now contains:

PostgreSQL Banking Source System

→ Python Ingestion Layer

→ Raw Parquet Zone

→ PySpark Standardized Layer

→ Curated Warehouse Layer

→ SQL Data Quality Validation

→ PySpark Data Quality Validation

→ Governance Artifacts

→ OpenMetadata Integration Layer

Final Phase 9 Result:

- Phase 9A completed and committed
- Phase 9B completed and committed
- Phase 9C completed and committed
- Metadata ingestion templates created

- Governance mappings documented
- Local deployment runbook created
- No secrets committed
- No pipeline logic modified
- Git history maintained
- Phase tagged as phase-9-complete

The project is now ready for Phase 10: Governance Operations and Stewardship Workflows.